Linear Methods for Regression: Prostate Cancer Example

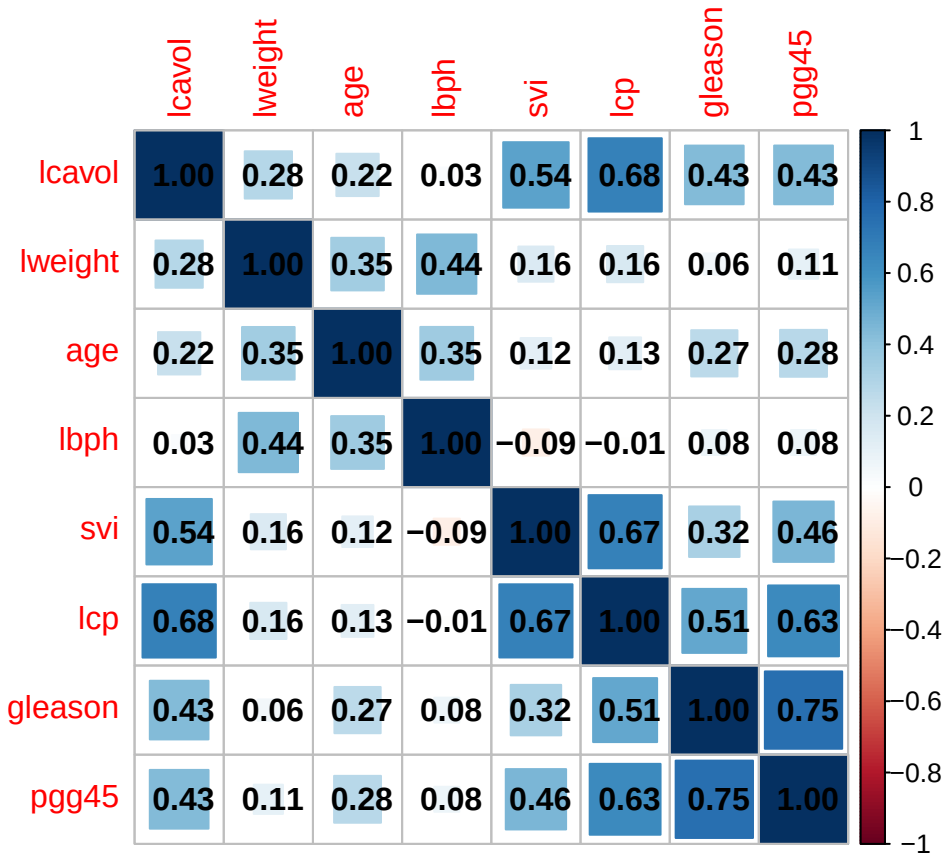
```
rm(list = ls())
library(stargazer) ; library(stats) ; library(tidyverse) ; library(readr)
library(corrplot) ; library(psych) ; library(glmnet)
set.seed(123) # set seed for reproducibility
# Prostate cancer data
# LEAST SQUARES -----------
# Data come from Stamey et al. (1989).
# Look a correlation between the level of prostate-specific antigen and
# a number of clinical measures in men who were about to receive a radical prostatectomy.
# The variables are :
# log cancer volume (lcavol)
# log prostate weight (lweight)
# age
# log of the amount of benign prostatic hyperplasia (lbph)
# seminal vesicle invasion (svi)
# log of capsular penetration (lcp),
# Gleason score (gleason)
# percent of Gleason scores 4 or 5 (pgg45).
# We first load the data and examine it.
data <- read_delim("prostate.data.txt", delim = "\t") # import the data
names(data)[1] <- "ID" # rename the ID column
head(data) ; summary(data) # visualize
```

```
## # A tibble: 6 x 11
##      ID lcavol         lweight   age lbph    svi lcp   gleason pgg45 lpsa  train
##   <dbl> <chr>            <dbl> <dbl> <chr> <dbl> <chr>   <dbl> <chr> <chr> <lgl>
## 1     1 "-0.579818495"    2.77    50 -1.3~     0 -1.3~       6 "  0" "-0.~ TRUE
## 2     2 "-0.994252273"    3.32    58 -1.3~     0 -1.3~       6 "  0" "-0.~ TRUE
## 3     3 "-0.510825624"    2.69    74 -1.3~     0 -1.3~       7 " 20" "-0.~ TRUE
## 4     4 "-1.203972804"    3.28    58 -1.3~     0 -1.3~       6 "  0" "-0.~ TRUE
## 5     5 " 0.751416089"    3.43    62 -1.3~     0 -1.3~       6 "  0" " 0.~ TRUE
## 6     6 "-1.049822124"    3.23    50 -1.3~     0 -1.3~       6 "  0" " 0.~ TRUE
```

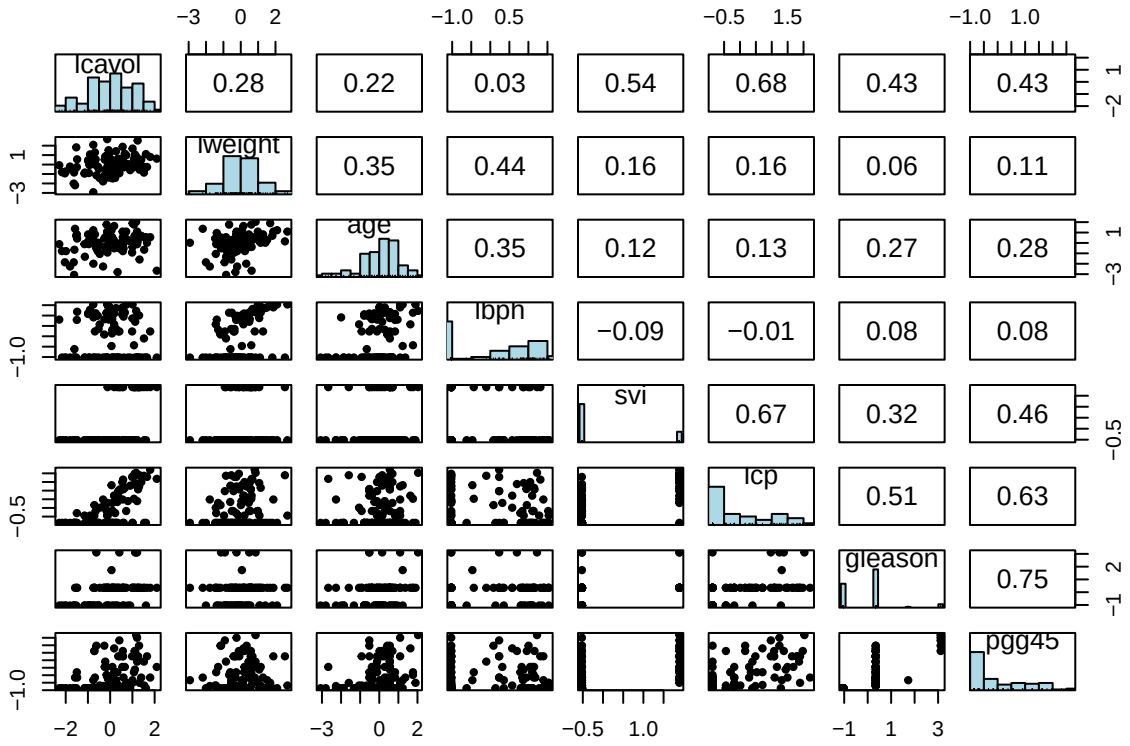
```
##        ID        lcavol            lweight           age       
##  Min.   : 1   Length:97          Min.   :2.375   Min.   :41.00  
##  1st Qu.:25   Class :character   1st Qu.:3.376   1st Qu.:60.00  
##  Median :49   Mode  :character   Median :3.623   Median :65.00  
##  Mean   :49                      Mean   :3.629   Mean   :63.87  
##  3rd Qu.:73                      3rd Qu.:3.876   3rd Qu.:68.00  
##  Max.   :97                      Max.   :4.780   Max.   :79.00  
##      lbph               svi              lcp              gleason     
##  Length:97          Min.   :0.0000   Length:97          Min.   :6.000  
##  Class :character   1st Qu.:0.0000   Class :character   1st Qu.:6.000  
##  Mode  :character   Median :0.0000   Mode  :character   Median :7.000  
##                     Mean   :0.2165                      Mean   :6.753  
##                     3rd Qu.:0.0000                      3rd Qu.:7.000  
##                     Max.   :1.0000                      Max.   :9.000
```

```
##     pgg45              lpsa              train
##  Length:97          Length:97          Mode :logical
##  Class :character   Class :character   FALSE:30
##  Mode  :character   Mode  :character   TRUE :67
##
##
##
```

```
vars <- c("lcavol", "lweight", "age", "lbph", "svi", "lcp", "gleason", "pgg45") # features
data <- data %>% mutate_at(append(vars, "lpsa"), as.numeric) # convert the the variables to numeric.
data <- data %>% mutate_at(vars, ~ scale(.) %>% as.vector) # standardize the features
corr_matrix <- cor(data[,vars]) # correlation matrix
corrplot(corr_matrix, method = "square", addCoef.col = 'black')  # associated plot
```




```
# scatter plot matrices (SPLOM),
# with bivariate scatter plots below the diagonal,
# histograms on the diagonal,
#and the Pearson correlation above the diagonal.
pairs.panels(data[, vars], hist.col = "lightblue", density = F, smooth = F,
             ellipses = F)
```




```
# We fit a linear model to the log of prostate-specific antigen, lpsa, after
# Randomly split the data into training set of size 67 and test set of size 30.
train_data <- subset(data, train == "TRUE") ; nrow(train_data)
```

```
## [1] 67
```

```
#train_data <- train_data %>% mutate_at(vars, ~ scale(.) %>% as.vector) # standardize the features
formula <- reformulate(vars, response = "lpsa") ; formula
```

```
## lpsa ~ lcavol + lweight + age + lbph + svi + lcp + gleason +
##     pgg45
```

```
model <- lm(formula, data = train_data)
summary(model)
```

```
##
## Call:
## lm(formula = formula, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.64870 -0.34147 -0.05424  0.44941  1.48675
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.46493    0.08931  27.598  < 2e-16 ***
## lcavol       0.67953    0.12663   5.366 1.47e-06 ***
## lweight      0.26305    0.09563   2.751  0.00792 **
## age         -0.14146    0.10134  -1.396  0.16806
## lbph         0.21015    0.10222   2.056  0.04431 *
## svi          0.30520    0.12360   2.469  0.01651 *
## lcp         -0.28849    0.15453  -1.867  0.06697 .
```

```
## gleason     -0.02131    0.14525  -0.147  0.88389
## pgg45        0.26696    0.15361   1.738  0.08755 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7123 on 58 degrees of freedom
## Multiple R-squared:  0.6944, Adjusted R-squared:  0.6522
## F-statistic: 16.47 on 8 and 58 DF,  p-value: 2.042e-12
```

```
# we compute the associated MSPE on the test data
test_data <- subset(data, train != "TRUE") ; nrow(test_data)
```

```
## [1] 30
```

```
#test_data <- test_data %>% mutate_at(vars, ~ scale(.) %>% as.vector) # standardize the features
predictions <- predict(model, newdata = test_data)
MSPE_1 <- sum((predictions - test_data$lpsa)^2)/nrow(test_data) ; MSPE_1 # approx 0.52
```

```
## [1] 0.521274
```

```
std_1 <- sqrt(var((predictions - test_data$lpsa)^2)/nrow(test_data)) ; std_1 # std deviation
```

```
## [1] 0.178724
```

```
# In contrast, if we use the mean lpsa in the training set,
# we would get a MSPE of
MSPE_2 <- sum((mean(train_data$lpsa) - test_data$lpsa)^2) / nrow(test_data) ; MSPE_2 # 1.06
```

```
## [1] 1.056733
```

```
MSPE_1/MSPE_2 - 1 # linear model reduces "base error rate" by about 50\%
```

```
## [1] -0.5067118
```

```
# RIDGE ----------
X <- model.matrix(formula, data = train_data)[,-1] # design matrix from train data
y <- train_data$lpsa # outcome vector from train data
ridge <- cv.glmnet(X, y, nfolds = 10, trace.it = 1, alpha = 0) # Ridge with CV10
```

```
## Training
##   |                                                                              |
```

```
## Fold: 10/10
##   |                                                                      |
```

```
coef(ridge, s = "lambda.min") # look at the model associated to lambda minimizing CV10
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##              lambda.min
## (Intercept)  2.46700218
## lcavol       0.58065709
## lweight      0.25757270
## age         -0.11032355
## lbph         0.20016172
## svi          0.28122176
## lcp         -0.16310975
## gleason      0.01246117
## pgg45        0.19962384
```

```
X_test <- model.matrix(formula, data = test_data)[,-1] # design matrix from test data
y_hat_ridge <- predict(ridge, newx = X_test, s = "lambda.min") # fitted values on test data
y_test <- test_data$lpsa # outcome vector from test data
MSPE_ridge <- sum((y_hat_ridge - y_test)^2) / nrow(test_data) ; MSPE_ridge # MSPE
```

```
## [1] 0.4943793
```

```
std_ridge  <- as.numeric(sqrt(var((y_hat_ridge - y_test)^2) / nrow(test_data))) ; std_ridge # sd
```

```
## [1] 0.1638058
```

```
MSPE_ridge / MSPE_1 - 1
```

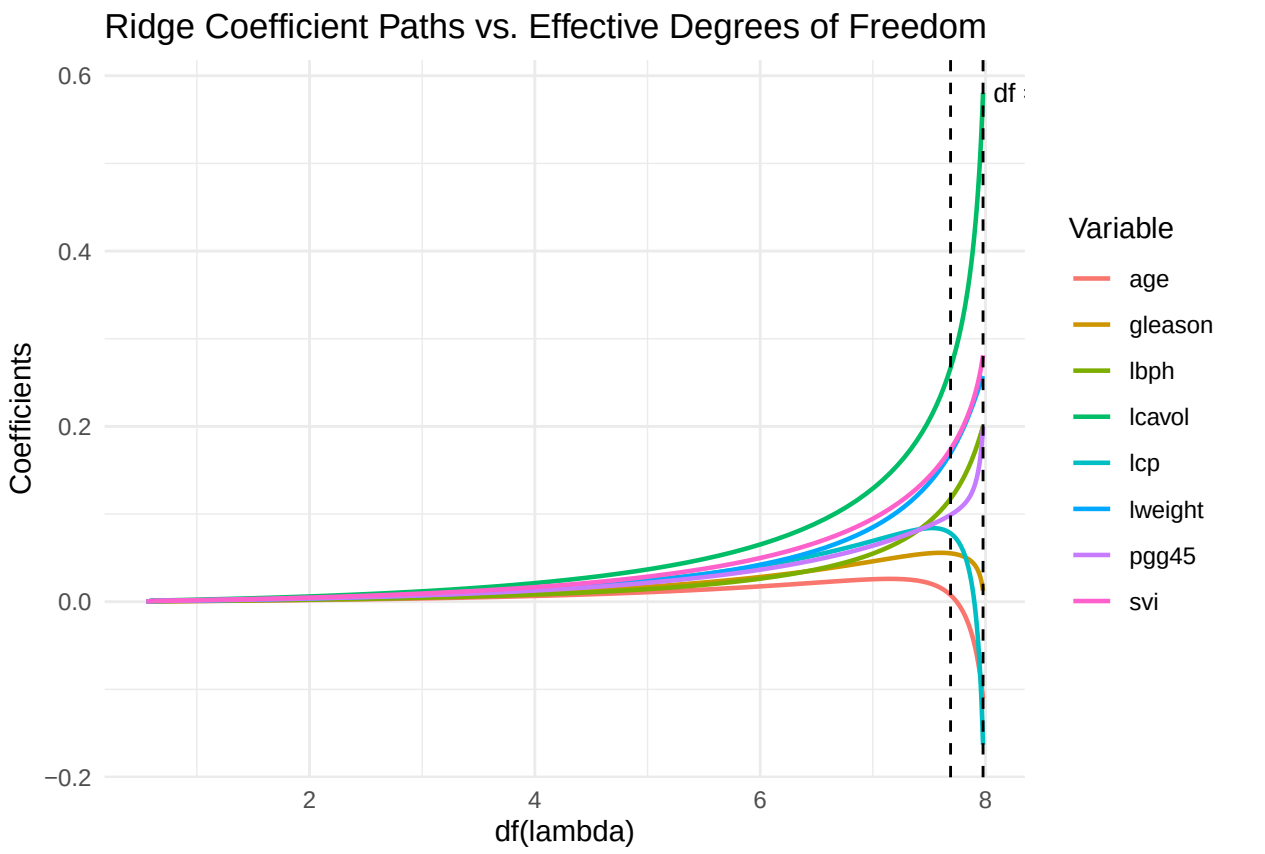
```
## [1] -0.05159409
```

```
# ridge reduces test error of LS estimates by a small amount. (around 5%)

df_lambda <- function(lambda, X){ # function to retrieve the effective df associated to each lambda
  hat_lambda <- X %*% solve(t(X) %*% X + lambda * diag(ncol(X))) %*% t(X)
  return(tr(hat_lambda))
}

lambdas <- ridge$glmnet.fit$lambda # the sequence of lambdas
coefs <- as.matrix(t(coef(ridge$glmnet.fit))) # coefs associated with each lambda value
coefs <- coefs[, -1]  # we don't care abobut the intercept
dfs <- sapply(lambdas, function(l) df_lambda(l, X)) # effective df at each lambda value
df_min <- df_lambda(ridge$lambda.min, X) # effective df at lambda min
df_1se <- df_lambda(ridge$lambda.1se, X) # effective df at lambda.1se
plot_data <- as.data.frame(coefs) # Reshape to long format for ggplot
plot_data$df <- dfs
plot_data_long <- plot_data %>%
  pivot_longer(cols = -df, names_to = "variable", values_to = "coefficient")
ggplot(plot_data_long, aes(x = df, y = coefficient, color = variable)) +
  geom_line(linewidth = 0.8) +
  geom_vline(xintercept = df_min, linetype = "dashed", color = "black") +
  geom_vline(xintercept = df_1se, linetype = "dashed", color = "black") +
  annotate("text", x = df_min, y = max(plot_data_long$coefficient),
           label = paste0("df = ", round(df_min, 2)), hjust = -0.1, size = 3.5) +
  labs(x = "df(lambda)", y = "Coefficients",
       title = "Ridge Coefficient Paths vs. Effective Degrees of Freedom",
```

```
       color = "Variable") +
  theme_minimal()
```

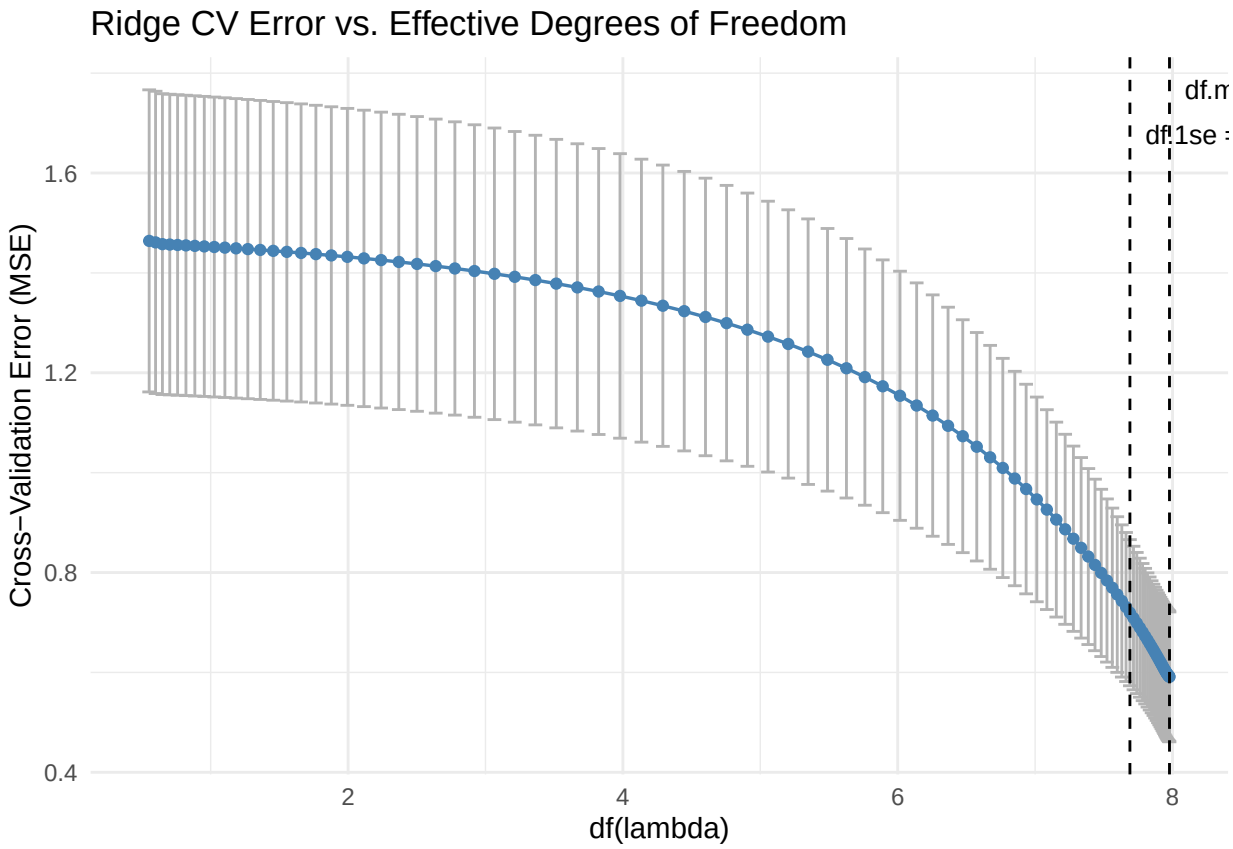



```
# the first dashed line is the effective df for lambda 1.se
# the second dashed line is the effective df for lambda.min.

# CV error data, aligned to the same lambda sequence as before
cv_data <- data.frame(
  lambda = ridge$lambda,
  df     = dfs,           # reuse the df vector you already computed
  cvm    = ridge$cvm,     # mean CV error at each lambda
  cvlo   = ridge$cvlo,    # lower bound of the CV error (mean - 1 SE)
  cvup   = ridge$cvup     # upper bound of the CV error (mean + 1 SE)
)

ggplot(cv_data, aes(x = df, y = cvm)) +
  geom_errorbar(aes(ymin = cvlo, ymax = cvup), color = "grey70", width = 0.1) +
  geom_line(color = "steelblue", linewidth = 0.6) +
  geom_point(color = "steelblue", size = 1.5) +
  geom_vline(xintercept = df_min, linetype = "dashed", color = "black") +
  geom_vline(xintercept = df_1se, linetype = "dashed", color = "black") +
  annotate("text", x = df_min, y = max(cv_data$cvup),
           label = paste0("df.min = ", round(df_min, 2)), hjust = -0.1, size = 3.5) +
  annotate("text", x = df_1se, y = max(cv_data$cvup) * 0.95,
           label = paste0("df.1se = ", round(df_1se, 2)), hjust = -0.1, size = 3.5) +
  labs(x = "df(lambda)", y = "Cross-Validation Error (MSE)",
       title = "Ridge CV Error vs. Effective Degrees of Freedom") +
```

```
  theme_minimal()
```




```
# LASSO ----------
lasso <- cv.glmnet(X, y, nfolds = 10, trace.it = 1, alpha = 1,
                   lambda = 10^seq(10, -2, length.out = 100)) # LASSO with CV10
```

```
## Training
##   |                                                                      |
```

```
##   |                                                                      |
```

```
coef(lasso, s = "lambda.1se") # look at the model associated to lambda minimizing CV10
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##             lambda.1se
## (Intercept) 2.46714942
## lcavol      0.53826934
## lweight     0.18532149
## age         .
## lbph        0.04563934
## svi         0.12600288
## lcp         .
## gleason     .
## pgg45       0.02632140
```

```
# using the least complex model within 1 standard error of the best model
y_hat_lasso <- predict(lasso, newx = X_test, s = "lambda.1se") # fitted values on test data
MSPE_lasso <- sum((y_hat_lasso - y_test)^2) / nrow(test_data) ; MSPE_lasso # MSPE
```

```
## [1] 0.4596582
```

```
std_lasso  <- as.numeric(sqrt(var((y_hat_lasso - y_test)^2) / nrow(test_data))) ; std_lasso # sd
```

```
## [1] 0.1571987
```

```
MSPE_lasso / MSPE_1 - 1 # lasso reduces test error of LS estimates by a bigger amount. (around 7.5%)
```

```
## [1] -0.1182024
```

```
MSPE_lasso / MSPE_ridge - 1 # lasso reduces test error of Ridge estimates by a small amount (around 3%)
```

```
## [1] -0.07023182
```

```
ols_fit  <- lm(y ~ X) ; summary(ols_fit)
```

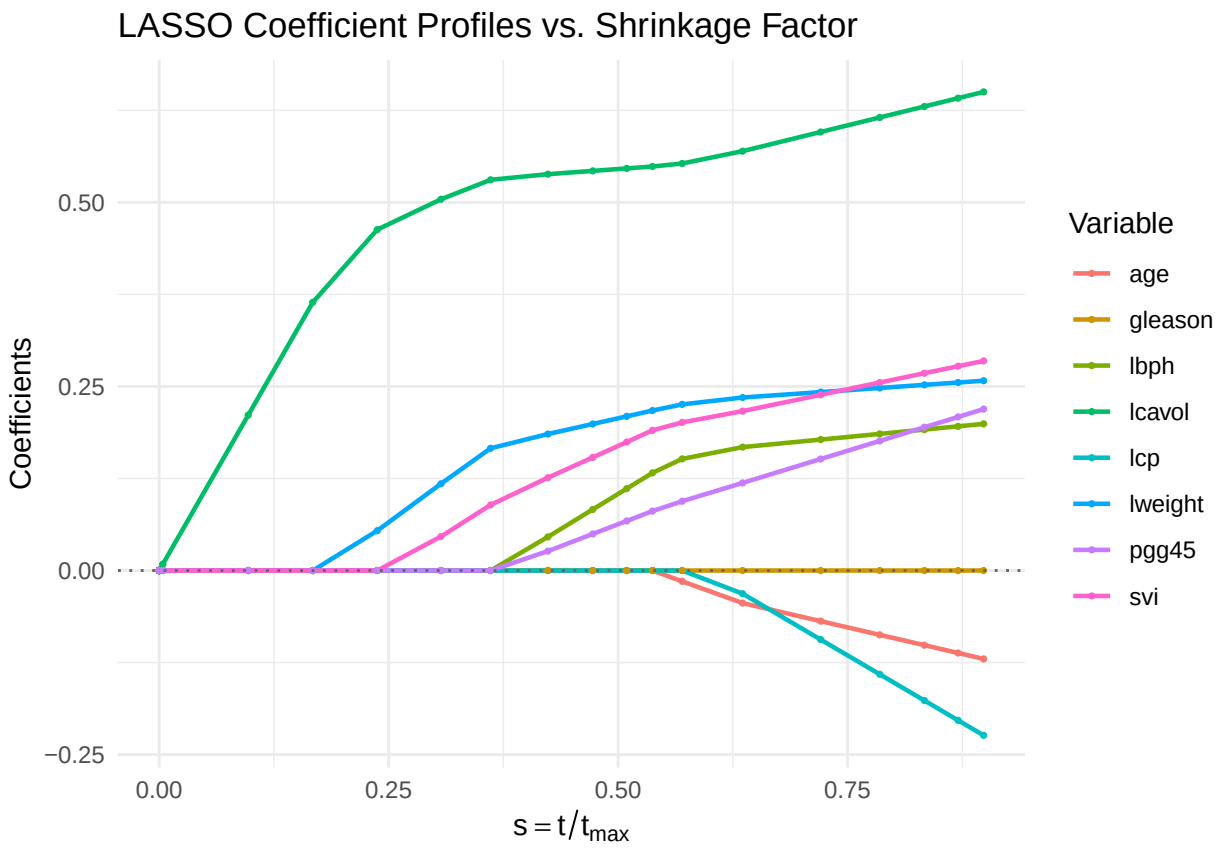
```
##
## Call:
## lm(formula = y ~ X)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.64870 -0.34147 -0.05424  0.44941  1.48675
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.46493    0.08931  27.598  < 2e-16 ***
## Xlcavol      0.67953    0.12663   5.366 1.47e-06 ***
## Xlweight     0.26305    0.09563   2.751  0.00792 **
## Xage        -0.14146    0.10134  -1.396  0.16806
## Xlbph        0.21015    0.10222   2.056  0.04431 *
## Xsvi         0.30520    0.12360   2.469  0.01651 *
## Xlcp        -0.28849    0.15453  -1.867  0.06697 .
## Xgleason    -0.02131    0.14525  -0.147  0.88389
## Xpgg45       0.26696    0.15361   1.738  0.08755 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 0.7123 on 58 degrees of freedom
## Multiple R-squared:  0.6944, Adjusted R-squared:  0.6522
## F-statistic: 16.47 on 8 and 58 DF,  p-value: 2.042e-12
```

```r
beta_ols <- coef(ols_fit)[-1]        # drop intercept
denom    <- sum(abs(beta_ols))       # max(t)

coefs_lasso <- as.matrix(t(coef(lasso$glmnet.fit)))[, -1]  # drop intercept column
t_lambda    <- rowSums(abs(coefs_lasso))                   # t at each lambda
s           <- t_lambda / denom                            # shrinkage factor
df_plot <- as.data.frame(coefs_lasso)
df_plot$s <- s
df_plot_long <- df_plot %>%
  pivot_longer(cols = -s, names_to = "variable", values_to = "coefficient")

ggplot(df_plot_long, aes(x = s, y = coefficient, color = variable)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 0.6) +
  geom_hline(yintercept = 0, linetype = "dotted", color = "grey40") +
  labs(x = expression(s == t / t[max]), y = "Coefficients",
       title = "LASSO Coefficient Profiles vs. Shrinkage Factor",
       color = "Variable") +
  theme_minimal()
```




```r
s_cv <- rowSums(abs(coefs_lasso)) / denom  # same s as before, one value per lambda in the path

cv_data_lasso <- data.frame(
  lambda = lasso$lambda,
```

```r
  s      = s_cv,
  cvm    = lasso$cvm, # mean Cv
  cvlo   = lasso$cvlo, # mean Cv - se
  cvup   = lasso$cvup # mean cv + se
)

s_min <- sum(abs(coef(lasso, s = "lambda.min")[-1])) / denom
s_1se <- sum(abs(coef(lasso, s = "lambda.1se")[-1])) / denom

ggplot(cv_data_lasso, aes(x = s, y = cvm)) +
  geom_errorbar(aes(ymin = cvlo, ymax = cvup), color = "grey70", width = 0.01) +
  geom_line(color = "steelblue", linewidth = 0.6) +
  geom_point(color = "steelblue", size = 1.5) +
  geom_vline(xintercept = s_min, linetype = "dashed", color = "black") +
  geom_vline(xintercept = s_1se, linetype = "dashed", color = "black") +
  annotate("text", x = s_min, y = max(cv_data_lasso$cvup),
           label = paste0("s.min = ", round(s_min, 2)), hjust = -0.1, size = 3.5) +
  annotate("text", x = s_1se, y = max(cv_data_lasso$cvup) * 0.95,
           label = paste0("s.1se = ", round(s_1se, 2)), hjust = -0.1, size = 3.5) +
  labs(x = expression(s == t / t[max]), y = "Cross-Validation Error (MSE)",
       title = "LASSO CV Error vs. Shrinkage Factor") +
  theme_minimal()
```